## Vectors

| Operation | Code | Time complexity |
|---|---|---|
| Push front | `v.insert(v.begin(), x)` | $O(n)$ |
| Pop front | `v.erase(v.begin())` | $O(n)$ |
| Push back | `v.push_back(x)` | $O(1)$ |
| Pop back | `v.pop_back()` | $O(1)$ |

# 1. Searching

## A. Linear search

Check elements one by one. It works on sorted and unsorted arrays.

```
int search(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}
```

Best case O(1); worst case O(n); extra space O(1).

Returns an index, not the value. -1 is a common “not found” marker.

With duplicates, this returns the first occurrence encountered.

## B. Binary search

Binary search halves the search range each step. The standard version requires a sorted array.

```
int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
```

```
    }
    return -1;
}
```

Time O(log n); iterative extra space O(1).

low <= high allows checking a range with one remaining element.

mid + 1 and mid - 1 exclude the already-checked middle element.

low + (high-low)/2 avoids potential overflow from (low+high)/2.

Standard code returns any matching duplicate, not necessarily the first or last. Descending arrays need adapted comparisons.

### Linear vs binary search

| Feature | Linear | Binary |
|---|---|---|
| Sorted input required? | No | Yes, standard version |
| Worst-case time | O(n) | O(log n) |
| Duplicates | First encountered | Any matching occurrence |
| Best use | Simple/unsorted data | Large sorted data |

Dry run: [2, 5, 8, 12, 16],

target 12. low=0, high=4, mid=2 (8);

since 8<12, low=3

. Next mid=3, target found at index 3.

## 2. Sorting

Sorting arranges values in order.

https://www.geeksforgeeks.org/dsa/sorting-algorithms/

Visualise Sorting:

https://visualgo.net/en/sorting

Imp Sorting Algos:

Comparison Based : Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort

## Bubble Sort

The rule is simple: compare adjacent elements; if the left one is bigger, swap them.

### The code

```
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - 1 - i; j++) {
        if (arr[j] > arr[j + 1]) {
            swap(arr[j], arr[j + 1]);
        }
    }
}
```

## Understand each line

- `i` = which pass we're on.
- `j` = which pair of neighbouring elements we're comparing.
- `arr[j]` and `arr[j+1]` = the neighbouring elements.
- `if (arr[j] > arr[j+1])` = swap only when they're in the wrong order.
- `n - 1 - i` = each pass leaves one more largest element fixed at the end, so we don't need to check it again.

## Selection sort

Find the minimum in the unsorted suffix and swap it into the first unsorted position.

```
for (int i = 0; i < n - 1; i++) {
    int minIndex = i;
    for (int j = i + 1; j < n; j++) {
        if (arr[j] < arr[minIndex]) minIndex = j;
    }
    swap(arr[i], arr[minIndex]);
}
```

Best/average/worst time $O(n^2)$; extra space $O(1)$.

Usually not stable in this swap-based implementation.

Outer loop fixes a position; inner loop searches for the smallest remaining value.

## Insertion sort

Treat the left side as sorted. Save the next key, shift larger values right, then insert the key in the gap.

```
for (int i = 1; i < n; i++) {
    int key = arr[i];
    int j = i - 1;
    while (j >= 0 && arr[j] > key) {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}
```

Best O(n) when already sorted; average/worst O(n²); extra space O(1).

Stable with the strict > comparison shown; equal elements are not shifted past each other.

Save key because shifting may overwrite its original position. Check j >= 0 before arr[j].

| Feature | Bubble Sort | Selection Sort | Insertion Sort |
|---|---|---|---|
| Main idea | Compare adjacent elements and swap if they're in the wrong order | Find the minimum element and place it at the beginning of the unsorted part | Take an element and insert it into its correct position in the sorted part |
| How elements move | Larger elements gradually move to the end | The minimum remaining element moves to its correct position | Larger elements shift right to make space |
| Number of swaps / moves | Can perform many swaps | At most $n - 1$ swaps in the standard implementation | Uses shifts and insertions |
| Best-case time | $O(n^2)$* | $O(n^2)$ | $O(n)$ |

| | | | |
|---|---|---|---|
| Average-case time | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ |
| Worst-case time | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ |
| Extra space | $O(1)$ | $O(1)$ | $O(1)$ |
| Stable? | Yes, if only strictly out-of-order adjacent elements are swapped | Usually no, with the standard swapping implementation | Yes, with the standard shifting implementation |
| Best suited for | Learning adjacent comparisons and swaps | Learning minimum selection | Small or nearly sorted arrays |

Consider an array of 5 elements in reverse order.

- Selection Sort: Finds the minimum and swaps it into place once per pass. It needs at most $n - 1 = 4$ swaps.
- Bubble Sort: Swaps adjacent elements repeatedly. In reverse order, it needs $\frac{n(n-1)}{2} = 10$ swaps.

## Common sorting doubts

Selection repeatedly selects a minimum; insertion shifts values to place a key.

Insertion starts at i=1 because a one-element prefix is already sorted.

j >= 0 prevents accessing index -1 in insertion sort.

In real C++, std::sort is commonly used; hand-written sorts are learned for fundamentals.

## Practice questions

1. Linear search: find the first index of a target, or return -1.
2. Binary search: find a target in a sorted array.
3. Two pointers: find one pair with target sum in a sorted array.
4. Dry-run selection sort on [5, 2, 4, 1].
5. Dry-run insertion sort on [5, 2, 4, 1].